

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № N4
по дисциплине «Информационные технологии и программирование»
по теме: «Разработка веб-приложения на Spring Boot. Работа с данными»

Выполнил студент группы:
БПИ2401 Исламов Эмин Маратович
Проверил: Харрасов Камиль
Раисович

Москва 2026

Цель работы

Научиться подключать Spring Boot-приложение к PostgreSQL и реализовать слой данных через Spring Data JPA/Hibernate: сущности, репозитории, CRUD, связи, транзакции, валидацию и проверку через Postman.

Архитектура и модули проекта

Слои (слоистая архитектура)

Controller - принимает HTTP-запросы, отдаёт ответы (DTO).

Service - бизнес-логика: создание/обновление сущностей, работа с транзакциями, доп.правила (например, проставление sentAt).

Repository - доступ к данным, интерфейсы Spring Data JPA (JpaRepository).

Пакеты (структура)

org.example/controller, service, repository, model/{dto,entity,enums} и др.

Связи между модулями

controller зависит от service (вызывает методы сервиса).

service зависит от repository (CRUD) и entity (доменные объекты).

repository типизирован entity и использует enums.

dto используется на границе API (контроллер - клиент).

Ход работы

1) application.properties

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Назначение этих настроек описано в методичке.

2) Перечисления (enums)

```
package org.example.model.enums;

public enum NotificationChannel {

    EMAIL,

    SMS,

    PUSH,

    TELEGRAM

}
```

```
package org.example.model.enums;

public enum NotificationStatus {

    CREATED,

    SENT,

    FAILED

}
```

3) Entity: User

```
package org.example.model.entity;
```

```

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;
import java.util.List;
import java.util.ArrayList;

@Getter
@Setter
@NoArgsConstructor
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name;
    private String email;
    private String phone;
    private String deviceToken;
    private String telegramChatId;
    private LocalDateTime createdAt;
    @OneToMany(mappedBy = "recipient", cascade =
CascadeType.ALL)
    private List<Notification> notifications = new
ArrayList<>();

```

```
}
```

4) Entity: Notification

```
package org.example.model.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import org.example.model.enums.NotificationChannel;
import org.example.model.enums.NotificationStatus;
import java.time.LocalDateTime;

@Getter
@Setter
@NoArgsConstructor
@Entity
@Table(name = "notification")
public class Notification {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String message;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
```

```

    private NotificationChannel channel;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private NotificationStatus status;

    private LocalDateTime createdAt;

    private LocalDateTime sentAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "recipient_id", nullable = false)
    private User recipient;
}

```

5) DTO: UserDto (с валидацией)

```

package org.example.model.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import java.time.LocalDateTime;

@Getter
@Setter
@Builder

```

```

@NoArgsConstructor
@AllArgsConstructor
public class UserDto {
    @NotBlank(message = "Имя не должно быть пустым")
    @Size(max = 100, message = "Имя не должно быть длинее
100 СИМВОЛОВ")
    private String name;
    @NotBlank(message = "Email обязателен")
    @Email(message = "Некорректный формат email")
    @Pattern(
        regexp = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$",
        message = "Email не соответствует требуемому
шаблону"
    )
    private String email;
    @Pattern(
        regexp = "^\\+7\\d{10}$",
        message = "Телефон должен быть в формате
+7xxxxxxxxxx"
    )
    private String phone;
    private String deviceToken;
    private String telegramChatId;
    private LocalDateTime createdAt;
}

```

Основа DTO и пример валидации в методичке.

6) DTO: NotificationDto

```

package org.example.model.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.*;

import org.example.model.enums.NotificationChannel;
import org.example.model.enums.NotificationStatus;
import org.example.service.NotificationManager;
import java.time.LocalDateTime;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor

public class NotificationDto {

    @NotBlank(message = "Заголовок не должен быть пустым")
    private String title;

    @NotBlank(message = "Текст сообщения не должен быть
пустым")
    private String message;

    @NotNull(message = "Канал обязателен")
    private NotificationChannel channel;

    private NotificationStatus status;

    private LocalDateTime createdAt;

    private LocalDateTime sentAt;

    @NotNull(message = "ИД получателя обязателен")
    private Long recipientId;
}

```


7) Репозитории

```
package org.example.repository;

import org.example.model.entity.User;
import
org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UserRepository extends JpaRepository<User,
Long> {
}
```

NotificationRepository (база + самостоятельные улучшения)

```
package org.example.repository;

import java.util.List;
import org.example.model.entity.Notification;
import org.example.model.enums.*;
import
org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

@Repository
public interface NotificationRepository extends
JpaRepository<Notification, Long> {

    List<Notification> findByStatus(NotificationStatus
status);
```

```

    List<Notification> findByChannel(NotificationChannel
channel);

    List<Notification> findByRecipientId(Long recipientId);

    List<Notification>
findByStatusAndChannel(NotificationStatus status,
NotificationChannel channel);

    List<Notification> findAllOrderByCreatedAtDesc();

    @Query("
        select n
        from Notification n
        where n.recipient.id = :recipientId
        and n.status = :status
    ")

    List<Notification> findByRecipientIdAndStatus(
        @Param("recipientId") Long recipientId,
        @Param("status") NotificationStatus status
    );

    @Query(value = "
        select *
        from notifications
        where status = :status
        and channel = :channel
    ")
    List<Notification>
findByStatusAndChannel(@Param("status") String
status,
nativeQuery = true)

```

```
@Param("channel") String  
channel);  
}
```

Базовые методы и идеи по нескольким параметрам/сортировке/@Query в методичке.

8) Service: UserService

```
package org.example.service;  
  
import java.util.List;  
import org.example.model.dto.*;  
import org.example.model.entity.*;  
import lombok.RequiredArgsConstructor;  
import org.example.repository.UserRepository;  
import org.springframework.stereotype.Service;  
  
@Service  
@RequiredArgsConstructor  
public class UserService {  
  
    private final UserRepository userRepository;  
  
    public User createUser(UserDto request){  
  
        User user = new User();  
        user.setName(request.getName());  
        user.setEmail(request.getEmail());  
        user.setPhone(request.getPhone());  
        user.setDeviceToken(request.getDeviceToken());  
        user.setTelegramChatId(request.getTelegramChatId());  
        user.setCreatedAt(request.getCreatedAt());  
  
        return userRepository.save(user);  
    }  
}
```

```

    }

    public List<User> getAllUsers(){
        return userRepository.findAll();
    }

    public User getUserByID(Long id){
        return userRepository.findById(id).orElseThrow();
    }

    public User updateUser(Long id, UserDto request){
        User user =
userRepository.findById(id).orElseThrow();
        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPhone(request.getPhone());
        user.setDeviceToken(request.getDeviceToken());
        user.setTelegramChatId(request.getTelegramChatId());
        return userRepository.save(user);
    }

    public void deleteUser(Long id){
        User user =
userRepository.findById(id).orElseThrow();
        userRepository.delete(user);
    }
}

```

9) Service: NotificationService (CRUD + транзакции + sentAt)

```
package org.example.service;

import org.springframework.web.server.ResponseStatusException;
import org.springframework.http.HttpStatus;
import jakarta.transaction.Transactional;
import lombok.RequiredArgsConstructor;
import org.example.model.dto.NotificationDto;
import org.example.model.entity.Notification;
import org.example.model.entity.User;
import org.example.model.enums.NotificationChannel;
import org.example.model.enums.NotificationStatus;
import org.example.repository.NotificationRepository;
import org.example.repository.UserRepository;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;

@Service
@RequiredArgsConstructor
public class NotificationService {

    private final NotificationRepository notificationRepository;

    private final UserRepository userRepository;

    @Transactional
    public Notification createNotification(NotificationDto request) {

        User user =
        userRepository.findById(request.getRecipientId()).orElseThr
```

```

ow() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
"User not found: " + request.getRecipientId()));

    Notification notification = new Notification();
    notification.setTitle(request.getTitle());
    notification.setMessage(request.getMessage());
    notification.setChannel(request.getChannel());
    notification.setStatus(NotificationStatus.CREATED);
    notification.setCreatedAt(LocalDateDateTime.now());
    notification.setRecipient(user);
    notificationRepository.save(notification);
    /* if (true) {
        throw new RuntimeException("Искусственная
ошибка");
    }
    */
    return notification;
}

public List<Notification> getAllNotifications() {
    return notificationRepository.findAll();
}

public Notification getNotificationById(Long id) {
    return notificationRepository.findById(id).orElseThrow();
}

public Notification updateNotification(Long id,
NotificationDto request) {
    Notification notification =
notificationRepository.findById(id).orElseThrow();

```

```

        notification.setTitle(request.getTitle());
        notification.setMessage(request.getMessage());
        notification.setChannel(request.getChannel());
        if (request.getStatus() != null) {
            notification.setStatus(request.getStatus());
            if (request.getStatus() ==
NotificationStatus.SENT && notification.getSentAt() ==
null) {
                notification.setSentAt(LocalDate.now());
            }
        }
        return notificationRepository.save(notification);
    }

    public void deleteNotification(Long id) {
        Notification notification =
notificationRepository.findById(id).orElseThrow();
        notificationRepository.delete(notification);
    }

    public List<Notification>
getNotificationsByStatus(NotificationStatus status) {
        return notificationRepository.findByStatus(status);
    }

    public List<Notification>
getNotificationsByChannel(NotificationChannel channel) {
        return
notificationRepository.findByChannel(channel);
    }

    public List<Notification>
getNotificationsByRecipientId(Long recipientId) {

```

```

        return
notificationRepository.findByRecipientId(recipientId);
    }
}

```

10) Controllers: UserController и NotificationController (DTO + @Valid + endpoints)

```

@RestController
@RequestMapping("/notifications")
@RequiredArgsConstructor
public class NotificationController {

    private final NotificationService notificationService;

    private NotificationDto mapToDto(Notification n) {
        return NotificationDto.builder()
            .title(n.getTitle())
            .message(n.getMessage())
            .channel(n.getChannel())
            .status(n.getStatus())
            .createdAt(n.getCreatedAt())
            .sentAt(n.getSentAt())
            .recipientId(n.getRecipient().getId())
            .build();
    }

    @PostMapping("/add")
}

```



```

    public NotificationDto createNotification(@RequestBody
@Valid
NotificationDto request) {

        Notification response =
notificationService.createNotification(request);

        return mapToDto(response);
    }

    @GetMapping("/all")

    public List<NotificationDto> getAllNotifications() {

        return
notificationService.getAllNotifications().stream().map(this
::mapToDto).toList();
    }

    @GetMapping("/{id}")

    public NotificationDto
getNotificationById(@PathVariable("id") Long id) {

        Notification response =
notificationService.getNotificationById(id);

        return mapToDto(response);
    }

    @PutMapping("/{id}")

    public NotificationDto
updateNotification(@PathVariable("id") Long id,
                                                             @RequestBody
@Valid
NotificationDto request) {

        Notification response =
notificationService.updateNotification(id,
request);

        return mapToDto(response);
    }

```

```

    }

    @DeleteMapping("/{id}")
    public String deleteNotification(@PathVariable("id")
Long id) {
        notificationService.deleteNotification(id);
        return "Уведомление удалено";
    }

    @GetMapping("/status/{status}")
    public List<NotificationDto>
getByStatus(@PathVariable("status")
NotificationStatus status) {
        return
notificationService.getNotificationsByStatus(status).stream
().map(this::mapToDto).toList();
    }

    @GetMapping("/channel/{channel}")
    public List<NotificationDto>
getByChannel(@PathVariable("channel")
NotificationChannel channel) {
        return
notificationService.getNotificationsByChannel(channel).stre
am().map(this::mapToDto).toList();
    }

    @GetMapping("/recipient/{recipientId}")
    public List<NotificationDto>
getByRecipientId(@PathVariable("recipientId") Long
recipientId) {
        return
notificationService.getNotificationsByRecipientId(recipient
Id).stream().map(this::mapToDto).toList();
    }

```

```
}  
}
```

```
@RestController  
@RequestMapping("/users")  
@RequiredArgsConstructor  
public class UserController {  
    private final UserService userService;  
    @PostMapping("/add")  
    public UserDto createUser(@RequestBody @Valid UserDto  
request) {  
        User response = userService.createUser(request);  
        return UserDto.builder()  
            .name(response.getName())  
            .email(response.getEmail())  
            .phone(response.getPhone())  
            .telegramChatId(response.getTelegramChatId(  
))  
            .deviceToken(response.getDeviceToken())  
            .createdAt(response.getCreatedAt())  
            .build();  
    }  
    @GetMapping("/all")  
    public List<UserDto> getAllUsers() {  
        return userService.getAllUsers().stream()  
            .map(r -> UserDto.builder()  
                .name(r.getName())
```

```

        .email(r.getEmail())
        .phone(r.getPhone())
        .telegramChatId(r.getTelegramChatId
    ))
    .deviceToken(r.getDeviceToken())
    .createdAt(r.getCreatedAt())
    .build()
    .toList();
}

@GetMapping("/{id}")
public UserDto getUserById(@PathVariable("id") Long id)
{
    User response = userService.getUserById(id);
    return UserDto.builder()
        .name(response.getName())
        .email(response.getEmail())
        .phone(response.getTelegramChatId())
        .deviceToken(response.getDeviceToken())
        .createdAt(response.getCreatedAt())
        .build();
}

@PutMapping("/{id}")
public UserDto updateUser(@PathVariable("id") Long id,
@RequestBody @Valid UserDto request){
    User response = userService.updateUser(id,
request);
    return UserDto.builder()
        .name(response.getName())

```

```

        .email(response.getEmail())
        .phone(response.getTelegramChatId())
        .deviceToken(response.getDeviceToken())
        .createdAt(response.getCreatedAt())
        .build();
    }

    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable("id") Long id){
        userService.deleteUser(id);
        return "User deleted";
    }
}

```

Как что связано

1. Клиент (Postman) отправляет JSON -> Controller принимает @RequestBody @Valid DTO.
2. Controller вызывает Service.
3. Service:
 - при create: загружает User по recipientId, создаёт Notification, ставит status=CREATED, createdAt=now, сохраняет в БД.
 - при update: меняет поля; если статус становится SENT -> ставит sentAt.
4. Repository (JpaRepository) генерирует SQL через Hibernate и выполняет.
5. Mapper переводит Entity -> DTO, чтобы наружу не отдавать сущности и связи.

Описание аннотаций

Spring / MVC

`@RestController` - контроллер, ответы автоматически сериализуются в JSON.

`@RequestMapping("/...")` - базовый путь контроллера.

`@PostMapping`, `@GetMapping`, `@PutMapping`, `@DeleteMapping` - маппинг HTTP-методов.

`@RequestBody` - тело запроса JSON -> объект.

`@PathVariable("id")` - параметр из пути `/.../{id}`.

`@Valid` - запускает Bean Validation; при ошибке отдаёт 400 и метод не выполняется.

DI / слои

`@Service` - компонент бизнес-логики.

`@Repository` - компонент доступа к данным.

`@RequiredArgsConstructor` - Lombok: конструктор под final поля (для DI).

JPA

`@Entity`, `@Table` - сущность и таблица.

`@Id`, `@GeneratedValue` - PK и стратегия генерации.

`@Column(nullable = false)` - ограничение NOT NULL.

`@Enumerated(EnumType.STRING)` - хранить enum как строку.

`@ManyToOne` + `@JoinColumn` - связь “многие к одному” (notification -> user).

@OneToMany(mappedBy=...) - обратная связь (user -> notifications).

Транзакции

@Transactional - гарантирует commit только при успехе, иначе rollback.

Валидация

@NotBlank, @NotNull, @Email, @Pattern, @Size - ограничения, при нарушении 400.

Вывод

В ходе лабораторной работы реализовано Spring Boot-приложение с доступом к данным через Spring Data JPA/Hibernate и PostgreSQL: созданы сущности User/Notification со связью one-to-many, DTO с валидацией, репозитории JpaRepository, CRUD-эндпоинты, транзакционность для атомарности операций, а также дополнительные методы поиска/сортировки и вынесен маппинг в отдельный mapper-класс.

Ответы на контрольные вопросы

1. Что такое Spring Data JPA и зачем он нужен?

Модуль Spring для упрощения работы с данными через JPA: репозитории на базе JpaRepository, автоматическая генерация CRUD и запросов по именам методов.

2. Какую роль выполняет Hibernate?

ORM-фреймворк (реализация JPA), отвечает за маппинг Java-объектов на таблицы, формирование SQL, связи и участие в транзакциях.

3. Что делает интерфейс JpaRepository?

Даёт готовые CRUD-методы (save, findAll, findById, delete) плюс сортировку/пагинацию и инфраструктуру для генерации запросов Spring Data.

4. Как реализуется связь один-ко-многим между User и Notification?

User содержит @OneToMany(mappedBy="recipient"), а Notification содержит @ManyToOne на User с FK recipient_id.

5. Что делает аннотация @ManyToOne?

Описывает связь “многие уведомления принадлежат одному пользователю”; в БД это внешний ключ в таблице notifications.

6. Как работает @Transactional?

Объединяет операции в одну транзакцию: commit только при успешном завершении; при исключении - rollback и данные не сохраняются частично.

7. В чем разница между JPQL и native SQL в @Query?

JPQL работает с сущностями/полями Java, native SQL - напрямую с таблицами/колонками БД.

8. Что делает @Valid в контроллере?

Запускает валидацию DTO перед вызовом метода; при нарушениях Spring возвращает 400 Bad Request.

9. Для чего используются @NotBlank, @Email, @Pattern?

Это аннотации Bean Validation для проверки строковых полей на пустоту, формат email и соответствие регулярному выражению.

10. Почему DTO удобнее валидировать, чем JPA-сущности напрямую?

DTO - граница API: можно валидировать вход клиента до БД, делать разные DTO для create/update, не смешивать правила внешнего ввода с моделью хранения и связями JPA.